

靠的方法是使用 GCC 支持的、操纵向量数据的 C 语言扩展。参见原书 546 页的网络旁注 OPT: SIMD, 看看可以怎么做到这样。

3.12 小结

在本章中, 我们窥视了 C 语言提供的抽象层下面的东西, 以了解机器级编程。通过让编译器产生机器级程序的汇编代码表示, 我们了解了编译器和它的优化能力, 以及机器、数据类型和指令集。在第 5 章, 我们会看到, 当编写能有效映射到机器上的程序时, 了解编译器的特性会有所帮助。我们还更完整地了解了程序如何将数据存储在不同的内存区域中。在第 12 章会看到许多这样的例子, 应用程序员需要知道一个程序变量是在运行时栈中, 是在某个动态分配的数据结构中, 还是全局程序数据的一部分。理解程序如何映射到机器上, 会让理解这些存储类型之间的区别容易一些。

机器级程序和它们的汇编代码表示, 与 C 程序的差别很大。各种数据类型之间的差别很小。程序是以指令序列来表示的, 每条指令都完成一个单独的操作。部分程序状态, 如寄存器和运行时栈, 对程序员来说是直接可见的。本书仅提供了低级操作来支持数据处理和程序控制。编译器必须使用多条指令来产生和操作各种数据结构, 以及实现像条件、循环和过程这样的控制结构。我们讲述了 C 语言和如何编译它的许多不同方面。我们看到 C 语言中缺乏边界检查, 使得许多程序容易出现缓冲区溢出。虽然最近的运行时系统提供了安全保护, 而且编译器帮助使得程序更安全, 但是这已经使许多系统容易受到恶意入侵者的攻击。

我们只分析了 C 到 x86-64 的映射, 但是大多数内容对其他语言和机器组合来说也是类似的。例如, 编译 C++ 与编译 C 就非常相似。实际上, C++ 的早期实现就只是简单地执行了从 C++ 到 C 的源到源的转换, 并对结果运行 C 编译器, 产生目标代码。C++ 的对象用结构来表示, 类似于 C 的 struct。C++ 的方法是用指向实现方法的代码的指针来表示的。相比而言, Java 的实现方式完全不同。Java 的目标代码是一种特殊的二进制表示, 称为 Java 字节代码。这种代码可以看成是虚拟机的机器级程序。正如它的名字暗示的那样, 这种机器并不是直接用硬件实现的, 而是用软件解释器处理字节代码, 模拟虚拟机的行为。另外, 有一种称为及时编译(just-in-time compilation)的方法, 动态地将字节代码序列翻译成机器指令。当代码要执行多次时(例如在循环中), 这种方法执行起来更快。用字节代码作为程序的低级表示, 优点是相同的代码可以在许多不同的机器上执行, 而在本章谈到的机器代码只能在 x86-64 机器上运行。

参考文献说明

Intel 和 AMD 提供了关于他们处理器的大量文档。包括从汇编语言程序员角度来看硬件的概貌[2, 50], 还包括每条指令的详细参考[3, 51]。读指令描述很复杂, 因为 1)所有的文档都基于 Intel 汇编代码格式, 2)由于不同的寻址和执行模式, 每条指令都有多个变种, 3)没有说明性示例。不过这些文档仍然是关于每条指令行为的权威参考。

组织 x86-64.org 负责定义运行在 Linux 系统上的 x86-64 代码的应用二进制接口(Application Binary Interface, ABI)[77]。这个接口描述了一些细节, 包括过程链接、二进制代码文件和大量的为了让机器代码程序正确运行所需要的其他特性。

正如我们讨论过的那样, GCC 使用的 ATT 格式与 Intel 文档中使用的 Intel 格式和其他编译器(包括 Microsoft 编译器)使用的格式都很不相同。

Muchnick 的关于编译器设计的书[80]被认为是关于代码优化技术最全面的参考书。它涵盖了许多我们在此讨论过的技术, 例如寄存器使用规则。

已经有很多文章是关于使用缓冲区溢出通过因特网来攻击系统的。Spafford 出版了关于 1988 年因特网蠕虫的详细分析[105], 而帮助阻止它传播的 MIT 团队的成员也出版了一些论著[35]。从那以后, 大量的论文和项目提出了各种创建和阻止缓冲区溢出攻击的方法。Seacord 的书[97]提供了关于缓冲区溢出和其他一些对 C 编译器产生的代码进行攻击的丰富信息。

家庭作业

- 3.58 一个函数的原型为

```
long decode2(long x, long y, long z);
```

GCC 产生如下汇编代码：

```
1  decode2:
2      subq    %rdx, %rsi
3      imulq   %rsi, %rdi
4      movq    %rsi, %rax
5      salq    $63, %rax
6      sarq    $63, %rax
7      xorq    %rdi, %rax
8      ret
```

参数 x 、 y 和 z 通过寄存器 $\%rdi$ 、 $\%rsi$ 和 $\%rdx$ 传递。代码将返回值存放在寄存器 $\%rax$ 中。

写出等价于上述汇编代码的 `decode2` 的 C 代码。

•• 3.59 下面的代码计算两个 64 位有符号值 x 和 y 的 128 位乘积，并将结果存储在内存中：

```
1  typedef __int128 int128_t;
2
3  void store_prod(int128_t *dest, int64_t x, int64_t y) {
4      *dest = x * (int128_t) y;
5  }
```

GCC 产出下面的汇编代码来实现计算：

```
1  store_prod:
2      movq    %rdx, %rax
3      cqto
4      movq    %rsi, %rcx
5      sarq    $63, %rcx
6      imulq   %rax, %rcx
7      imulq   %rsi, %rdx
8      addq    %rdx, %rcx
9      mulq    %rsi
10     addq    %rcx, %rdx
11     movq    %rax, (%rdi)
12     movq    %rdx, 8(%rdi)
13     ret
```

为了满足在 64 位机器上实现 128 位运算所需的多精度计算，这段代码用了三个乘法。描述用来计算乘积的算法，对汇编代码加注注释，说明它是如何实现你的算法的。提示：在把参数 x 和 y 扩展到 128 位时，它们可以重写为 $x = 2^{64} \cdot x_h + x_l$ 和 $y = 2^{64} \cdot y_h + y_l$ ，这里 x_h 、 x_l 、 y_h 和 y_l 都是 64 位值。类似地，128 位的乘积可以写成 $p = 2^{64} \cdot p_h + p_l$ ，这里 p_h 和 p_l 是 64 位值。请解释这段代码是如何用 x_h 、 x_l 、 y_h 和 y_l 来计算 p_h 和 p_l 的。

•• 3.60 考虑下面的汇编代码：

```
long loop(long x, int n)
x in %rdi, n in %esi
1  loop:
2      movl    %esi, %ecx
3      movl    $1, %edx
4      movl    $0, %eax
5      jmp     .L2
6  .L3:
7      movq    %rdi, %r8
8      andq    %rdx, %r8
9      orq     %r8, %rax
10     salq    %cl, %rdx
11  .L2:
12     testq   %rdx, %rdx
13     jne     .L3
14     rep; ret
```


以上代码是编译以下整体形式的 C 代码产生的：

```

1  long loop(long x, int n)
2  {
3      long result = _____;
4      long mask;
5      for (mask = _____; mask _____; mask = _____) {
6          result |= _____;
7      }
8      return result;
9  }
```

你的任务是填写这个 C 代码中缺失的部分，得到一个程序等价于产生的汇编代码。回想一下，这个函数的结果是在寄存器 %rax 中返回的。你会发现以下工作很有帮助：检查循环之前、之中和之后的汇编代码，形成一个寄存器和程序变量之间一致的映射。

- A. 哪个寄存器保存着程序值 x、n、result 和 mask?
- B. result 和 mask 的初始值是什么?
- C. mask 的测试条件是什么?
- D. mask 是如何被修改的?
- E. result 是如何被修改的?
- F. 填写这段 C 代码中所有缺失的部分。

**** 3.61** 在 3.6.6 节，我们查看了下面的代码，作为使用条件数据传送的一种选择：

```

long cread(long *xp) {
    return (xp ? *xp : 0);
}
```

我们给出了使用条件传送指令的一个尝试实现，但是认为它是不合法的，因为它试图从一个空地址读数据。

写一个 C 函数 `cread_alt`，它与 `cread` 有一样的行为，除了它可以被编译成使用条件数据传送。当编译时，产生的代码应该使用条件传送指令而不是某种跳转指令。

**** 3.62** 下面的代码给出了一个开关语句中根据枚举类型值进行分支选择的例子。回忆一下，C 语言中枚举类型只是一种引入一组与整数值相对应的名字的方法。默认情况下，值是从 0 向上依次赋给名字的。在我们的代码中，省略了与各种情况标号相对应的动作。

```

1  /* Enumerated type creates set of constants numbered 0 and upward */
2  typedef enum {MODE_A, MODE_B, MODE_C, MODE_D, MODE_E} mode_t;
3
4  long switch3(long *p1, long *p2, mode_t action)
5  {
6      long result = 0;
7      switch(action) {
8          case MODE_A:
9
10         case MODE_B:
11
12         case MODE_C:
13
14         case MODE_D:
15
16         case MODE_E:
17
18         default:
19
20     }
21     return result;
22 }
```

产生的实现各个动作的汇编代码部分如图 3-52 所示。注释指明了参数位置，寄存器值，以及各个跳转目的的情况标号。

```

    pl in %rdi, p2 in %rsi, action in %edx
1   .L8:                                MODE_E
2       movl    $27, %eax
3       ret
4   .L3:                                MODE_A
5       movq    (%rsi), %rax
6       movq    (%rdi), %rdx
7       movq    %rdx, (%rsi)
8       ret
9   .L5:                                MODE_B
10      movq    (%rdi), %rax
11      addq    (%rsi), %rax
12      movq    %rax, (%rdi)
13      ret
14   .L6:                                MODE_C
15      movq    $59, (%rdi)
16      movq    (%rsi), %rax
17      ret
18   .L7:                                MODE_D
19      movq    (%rsi), %rax
20      movq    %rax, (%rdi)
21      movl    $27, %eax
22      ret
23   .L9:                                default
24      movl    $12, %eax
25      ret

```

图 3-52 家庭作业 3.62 的汇编代码。这段代码实现了 switch 语句的各个分支

填写 C 代码中缺失的部分。代码包括落入其他情况的情况，试着重建这个情况。

3.63 这个程序给你一个机会，从反汇编机器代码逆向工程一个 switch 语句。在下面这个过程中，去掉了 switch 语句的主体：

```

1   long switch_prob(long x, long n) {
2       long result = x;
3       switch(n) {
4           /* Fill in code here */
5
6       }
7       return result;
8   }

```

图 3-53 给出了这个过程的反汇编机器代码。

跳转表驻留在内存的不同区域中。可以从第 5 行的间接跳转看出来，跳转表的起始地址为 0x4006f8。用调试器 GDB，我们可以用命令 `x/6gx 0x4006f8` 来检查组成跳转表的 6 个 8 字节字的内存。GDB 打印出下面的内容：

```

(gdb) x/6gx 0x4006f8
0x4006f8:  0x000000000004005a1  0x000000000004005c3
0x400708:  0x000000000004005a1  0x000000000004005aa
0x400718:  0x000000000004005b2  0x000000000004005bf

```

用 C 代码填写开关语句的主体，使它的行为与机器代码一致。

```

long switch_prob(long x, long n)
x in %rdi, n in %rsi
1 0000000000400590 <switch_prob>:
2 400590: 48 83 ee 3c          sub    $0x3c,%rsi
3 400594: 48 83 fe 05          cmp    $0x5,%rsi
4 400598: 77 29              ja     4005c3 <switch_prob+0x33>
5 40059a: ff 24 f5 f8 06 40 00 jmpq   *0x4006f8(,%rsi,8)
6 4005a1: 48 8d 04 fd 00 00 00 lea     0x0(,%rdi,8),%rax
7 4005a8: 00
8 4005a9: c3                retq
9 4005aa: 48 89 f8          mov    %rdi,%rax
10 4005ad: 48 c1 f8 03       sar    $0x3,%rax
11 4005b1: c3                retq
12 4005b2: 48 89 f8          mov    %rdi,%rax
13 4005b5: 48 c1 e0 04       shl    $0x4,%rax
14 4005b9: 48 29 f8          sub    %rdi,%rax
15 4005bc: 48 89 c7          mov    %rax,%rdi
16 4005bf: 48 0f af ff       imul   %rdi,%rdi
17 4005c3: 48 8d 47 4b       lea     0x4b(%rdi),%rax
18 4005c7: c3                retq

```

图 3-53 家庭作业 3.63 的反汇编代码

**** 3.64** 考虑下面的源代码，这里 *R*、*S* 和 *T* 都是用 `#define` 声明的常数：

```

1 long A[R][S][T];
2
3 long store_ele(long i, long j, long k, long *dest)
4 {
5     *dest = A[i][j][k];
6     return sizeof(A);
7 }

```

在编译这个程序中，GCC 产生下面的汇编代码：

```

long store_ele(long i, long j, long k, long *dest)
i in %rdi, j in %rsi, k in %rdx, dest in %rcx
1 store_ele:
2     leaq    (%rsi,%rsi,2), %rax
3     leaq    (%rsi,%rax,4), %rax
4     movq    %rdi, %rsi
5     salq    $6, %rsi
6     addq    %rsi, %rdi
7     addq    %rax, %rdi
8     addq    %rdi, %rdx
9     movq    A(,%rdx,8), %rax
10    movq    %rax, (%rcx)
11    movl    $3640, %eax
12    ret

```

A. 将等式 (3.1) 从二维扩展到三维，提供数组元素 $A[i][j][k]$ 的位置的公式。

B. 运用你的逆向工程技术，根据汇编代码，确定 *R*、*S* 和 *T* 的值。

*** 3.65** 下面的代码转置一个 $M \times M$ 矩阵的元素，这里 *M* 是一个用 `#define` 定义的常数：

```

1 void transpose(long A[M][M]) {
2     long i, j;
3     for (i = 0; i < M; i++)
4         for (j = 0; j < i; j++) {
5             long t = A[i][j];
6             A[i][j] = A[j][i];
7             A[j][i] = t;
8         }
9 }

```


当用优化等级 -O1 编译时, GCC 为这个函数的内循环产生下面的代码:

```

1  .L6:
2      movq    (%rdx), %rcx
3      movq    (%rax), %rsi
4      movq    %rsi, (%rdx)
5      movq    %rcx, (%rax)
6      addq    $8, %rdx
7      addq    $120, %rax
8      cmpq    %rdi, %rax
9      jne     .L6

```

我们可以看到 GCC 把数组索引转换成了指针代码。

- A. 哪个寄存器保存着指向数组元素 $A[i][j]$ 的指针?
- B. 哪个寄存器保存着指向数组元素 $A[j][i]$ 的指针?
- C. M 的值是多少?

• 3.66 考虑下面的源代码, 这里 NR 和 NC 是用 #define 声明的宏表达式, 计算用参数 n 表示的矩阵 A 的维度。这段代码计算矩阵的第 j 列的元素之和。

```

1  long sum_col(long n, long A[NR(n)][NC(n)], long j) {
2      long i;
3      long result = 0;
4      for (i = 0; i < NR(n); i++)
5          result += A[i][j];
6      return result;
7  }

```

编译这个程序, GCC 产生下面的汇编代码:

```

long sum_col(long n, long A[NR(n)][NC(n)], long j)
n in %rdi, A in %rsi, j in %rdx
1  sum_col:
2      leaq    1(,%rdi,4), %r8
3      leaq    (%rdi,%rdi,2), %rax
4      movq    %rax, %rdi
5      testq   %rax, %rax
6      jle     .L4
7      salq    $3, %r8
8      leaq    (%rsi,%rdx,8), %rcx
9      movl    $0, %eax
10     movl    $0, %edx
11  .L3:
12     addq    (%rcx), %rax
13     addq    $1, %rdx
14     addq    %r8, %rcx
15     cmpq    %rdi, %rdx
16     jne     .L3
17     rep; ret
18  .L4:
19     movl    $0, %eax
20     ret

```

运用你的逆向工程技术, 确定 NR 和 NC 的定义。

• 3.67 这个作业要查看 GCC 为参数和返回值中有结构的函数产生的代码, 由此可以看到这些语言特性通常是如何实现的。

下面的 C 代码中有一个函数 process, 它用结构作为参数和返回值, 还有一个函数 eval, 它调用 process:

```

1  typedef struct {
2      long a[2];

```

```

3     long *p;
4 } strA;
5
6 typedef struct {
7     long u[2];
8     long q;
9 } strB;
10
11 strB process(strA s) {
12     strB r;
13     r.u[0] = s.a[1];
14     r.u[1] = s.a[0];
15     r.q = *s.p;
16     return r;
17 }
18
19 long eval(long x, long y, long z) {
20     strA s;
21     s.a[0] = x;
22     s.a[1] = y;
23     s.p = &z;
24     strB r = process(s);
25     return r.u[0] + r.u[1] + r.q;
26 }

```

GCC 为这两个函数产生下面的代码：

```

      strB process(strA s)
1 process:
2     movq    %rdi, %rax
3     movq    24(%rsp), %rdx
4     movq    (%rdx), %rdx
5     movq    16(%rsp), %rcx
6     movq    %rcx, (%rdi)
7     movq    8(%rsp), %rcx
8     movq    %rcx, 8(%rdi)
9     movq    %rdx, 16(%rdi)
10    ret

      long eval(long x, long y, long z)
      x in %rdi, y in %rsi, z in %rdx
1 eval:
2     subq    $104, %rsp
3     movq    %rdx, 24(%rsp)
4     leaq    24(%rsp), %rax
5     movq    %rdi, (%rsp)
6     movq    %rsi, 8(%rsp)
7     movq    %rax, 16(%rsp)
8     leaq    64(%rsp), %rdi
9     call    process
10    movq    72(%rsp), %rax
11    addq    64(%rsp), %rax
12    addq    80(%rsp), %rax
13    addq    $104, %rsp
14    ret

```

- A. 从 eval 函数的第 2 行我们可以看到，它在栈上分配了 104 个字节。画出 eval 的栈帧，给出它在调用 process 前存储在栈上的值。
- B. eval 调用 process 时传递了什么值？
- C. process 的代码是如何访问结构参数 s 的元素的？
- D. process 的代码是如何设置结果结构 r 的字段的？

- E. 完成 eval 的栈帧图,给出在从 process 返回后 eval 是如何访问结构 r 的元素的。
 F. 就如何传递作为函数参数的结构以及如何返回作为函数结果的结构值,你可以看出什么通用的原则?

***3.68 在下面的代码中, A 和 B 是用 # define 定义的常数:

```

1  typedef struct {
2      int x[A][B]; /* Unknown constants A and B */
3      long y;
4  } str1;
5
6  typedef struct {
7      char array[B];
8      int t;
9      short s[A];
10     long u;
11 } str2;
12
13 void setVal(str1 *p, str2 *q) {
14     long v1 = q->t;
15     long v2 = q->u;
16     p->y = v1+v2;
17 }
```

GCC 为 setVal 产生下面的代码:

```

void setVal(str1 *p, str2 *q)
p in %rdi, q in %rsi
setVal:
1  movslq 8(%rsi), %rax
2  addq 32(%rsi), %rax
3
4  movq %rax, 184(%rdi)
5  ret
```

A 和 B 的值是多少? (答案是唯一的。)

***3.69 你负责维护一个大型的 C 程序,遇到下面的代码:

```

1  typedef struct {
2      int first;
3      a_struct a[CNT];
4      int last;
5  } b_struct;
6
7  void test(long i, b_struct *bp)
8  {
9      int n = bp->first + bp->last;
10     a_struct *ap = &bp->a[i];
11     ap->x[ap->idx] = n;
12 }
```

编译时常数 CNT 和结构 a_struct 的声明是在一个你没有访问权限的文件中。幸好,你有代码的 '.o' 版本,可以用 OBJDUMP 程序来反汇编这些文件,得到下面的反汇编代码:

```

void test(long i, b_struct *bp)
i in %rdi, bp in %rsi
1  0000000000000000 <test>:
2      0:  8b 8e 20 01 00 00      mov     0x120(%rsi),%ecx
3      6:  03 0e                  add     (%rsi),%ecx
4      8:  48 8d 04 bf          lea     (%rdi,%rdi,4),%rax
5      c:  48 8d 04 c6          lea     (%rsi,%rax,8),%rax
6     10:  48 8b 50 08          mov     0x8(%rax),%rdx
7     14:  48 63 c9          movslq %ecx,%rcx
```



```

8      17:  48 89 4c d0 10      mov    %rcx,0x10(%rax,%rdx,8)
9      1c:  c3                          retq

```

运用你的逆向工程技术，推断出下列内容：

- A. CNT 的值。
- B. 结构 `a_struct` 的完整声明。假设这个结构中只有字段 `idx` 和 `x`，并且这两个字段保存的都是有符号值。

**** 3.70** 考虑下面的联合声明：

```

1  union ele {
2      struct {
3          long *p;
4          long y;
5      } e1;
6      struct {
7          long x;
8          union ele *next;
9      } e2;
10 };

```

这个声明说明联合中可以嵌套结构。

下面的函数(省略了一些表达式)对一个链表进行操作，链表是以上述联合作为元素的：

```

1  void proc (union ele *up) {
2      up->_____ = *(_____) - _____;
3  }

```

- A. 下列字段的偏移量是多少(以字节为单位)：

```

e1.p      _____
e1.y      _____
e2.x      _____
e2.next   _____

```

- B. 这个结构总共需要多少个字节？
- C. 编译器为 `proc` 产生下面的汇编代码：

```

void proc (union ele *up)
up in %rdi
1  proc:
2      movq    8(%rdi), %rax
3      movq    (%rax), %rdx
4      movq    (%rdx), %rdx
5      subq    8(%rax), %rdx
6      movq    %rdx, (%rdi)
7      ret

```

在这些信息的基础上，填写 `proc` 代码中缺失的表达式。提示：有些联合引用的解释可以有歧义。当你清楚引用指引到哪里的时候，就能够澄清这些歧义。只有一个答案，不需要进行强制类型转换，且不违反任何类型限制。

- * 3.71** 写一个函数 `good_echo`，它从标准输入读取一行，再把它写到标准输出。你的实现应该对任意长度的输入行都能工作。可以使用库函数 `fgets`，但是你必须确保即使当输入行要求比你已经为缓冲区分配的更多的空间时，你的函数也能正确地工作。你的代码还应该检查错误条件，要在遇到错误条件时返回。参考标准 I/O 函数的定义文档[45，61]。

- ** 3.72** 图 3-54a 给出了一个函数的代码，该函数类似于函数 `vfunct`(图 3-43a)。我们用 `vfunct` 来说明过帧指针在管理变长栈帧中的使用情况。这里的新函数 `aframe` 调用库函数 `alloca` 为局部数组 `p` 分配空间。`alloca` 类似于更常用的函数 `malloc`，区别在于它在运行时栈上分配空间。当正在执行的过程返回时，该空间会自动释放。

图 3-54b 给出了部分的汇编代码，建立帧指针，为局部变量 `i` 和 `p` 分配空间。非常类似于

vframe 对应的代码。在此使用与练习题 3.49 中同样的表示法：栈指针在第 4 行设置为值 s_1 ，在第 7 行设置为值 s_2 。数组 p 的起始地址在第 9 行被设置为值 p 。 s_2 和 p 之间可能有额外的空间 e_2 ，数组 p 结尾和 s_1 之间可能有额外的空间 e_1 。

- A. 用数学语言解释计算 s_2 的逻辑。
- B. 用数学语言解释计算 p 的逻辑。
- C. 确定使 e_1 的值最小和最大的 n 和 s_1 的值。
- D. 这段代码为 s_2 和 p 的值保证了怎样的对齐属性？

```
1  #include <alloca.h>
2
3  long aframe(long n, long idx, long *q) {
4      long i;
5      long **p = alloca(n * sizeof(long *));
6      p[0] = &i;
7      for (i = 1; i < n; i++)
8          p[i] = q;
9      return *p[idx];
10 }
```

a) C代码

```
long aframe(long n, long idx, long *q)
n in %rdi, idx in %rsi, q in %rdx
1  aframe:
2      pushq    %rbp
3      movq    %rsp, %rbp
4      subq    $16, %rsp           Allocate space for i (%rsp = s1)
5      leaq    30(,%rdi,8), %rax
6      andq    $-16, %rax
7      subq    %rax, %rsp           Allocate space for array p (%rsp = s2)
8      leaq    15(%rsp), %r8
9      andq    $-16, %r8           Set %r8 to &p[0]
   :
   :
```

b) 部分生成的汇编代码

图 3-54 家庭作业 3.72 的代码。该函数类似于图 3-43 中的函数

- 3.73 用汇编代码写出匹配图 3-51 中函数 `find_range` 行为的函数。你的代码必须只包含一个浮点比较指令，并用条件分支指令来生成正确的结果。在 2^{32} 种可能的参数值上测试你的代码。网络旁注 ASM:EASM 描述了如何在 C 程序中嵌入汇编代码。
- 3.74 用汇编代码写出匹配图 3-51 中函数 `find_range` 行为的函数。你的代码必须只包含一个浮点比较指令，并用条件传送指令来生成正确的结果。你可能会想要使用指令 `cmovp` (如果设置了偶校验位传送)。在 2^{32} 种可能的参数值上测试你的代码。网络旁注 ASM:EASM 描述了如何在 C 程序中嵌入汇编代码。
- 3.75 ISO C99 包括了支持复数的扩展。任何浮点类型都可以用关键字 `complex` 修饰。这里有一些使用复数数据的示例函数，调用了一些关联的库函数：

```
1  #include <complex.h>
2
3  double c_imag(double complex x) {
4      return cimag(x);
5  }
6
7  double c_real(double complex x) {
8      return creal(x);
9  }
```



```
9  }
10
11  double complex c_sub(double complex x, double complex y) {
12      return x - y;
13  }
```

编译时，GCC 为这些函数产生如下代码：

```
double c_imag(double complex x)
1  c_imag:
2      movapd  %xmm1, %xmm0
3      ret

double c_real(double complex x)
4  c_real:
5      rep; ret

double complex c_sub(double complex x, double complex y)
6  c_sub:
7      subsd  %xmm2, %xmm0
8      subsd  %xmm3, %xmm1
9      ret
```

根据这些例子，回答下列问题：

- A. 如何向函数传递复数参数？
- B. 如何从函数返回复数值？

练习题答案

3.1 这个练习使你熟悉各种操作数格式。

操作数	值	注释
%rax	0x100	寄存器
0x104	0xAB	绝对地址
\$0x108	0x108	立即数
(%rax)	0xFF	地址 0x100
4(%rax)	0xAB	地址 0x104
9(%rax,%rdx)	0x11	地址 0x10C
260(\$rcx,%rdx)	0x13	地址 0x108
0XFC(,%rcx,4)	0xFF	地址 0x100
(%rax,%rdx,4)	0x11	地址 0x10C

3.2 正如我们已经看到的，GCC 产生的汇编代码指令上有后缀，而反汇编代码没有。能够在这两种形式之间转换是一种很重要的需要学习的技能。一个重要的特性就是，x86-64 中的内存引用总是用四字长寄存器给出，例如 %rax，哪怕操作数只是一个字节、一个字或是一个双字。

这里是带后缀的代码：

```
movl  %eax, (%rsp)
movw  (%rax), %dx
movb  $0xFF, %bl
movb  (%rsp,%rdx,4), %dl
movq  (%rdx), %rax
movw  %dx, (%rax)
```

3.3 由于我们会依赖 GCC 来产生大多数汇编代码，所以能够写正确的汇编代码并不是一项很关键的技能。但是，这个练习会帮助你熟悉不同的指令和操作数类型。